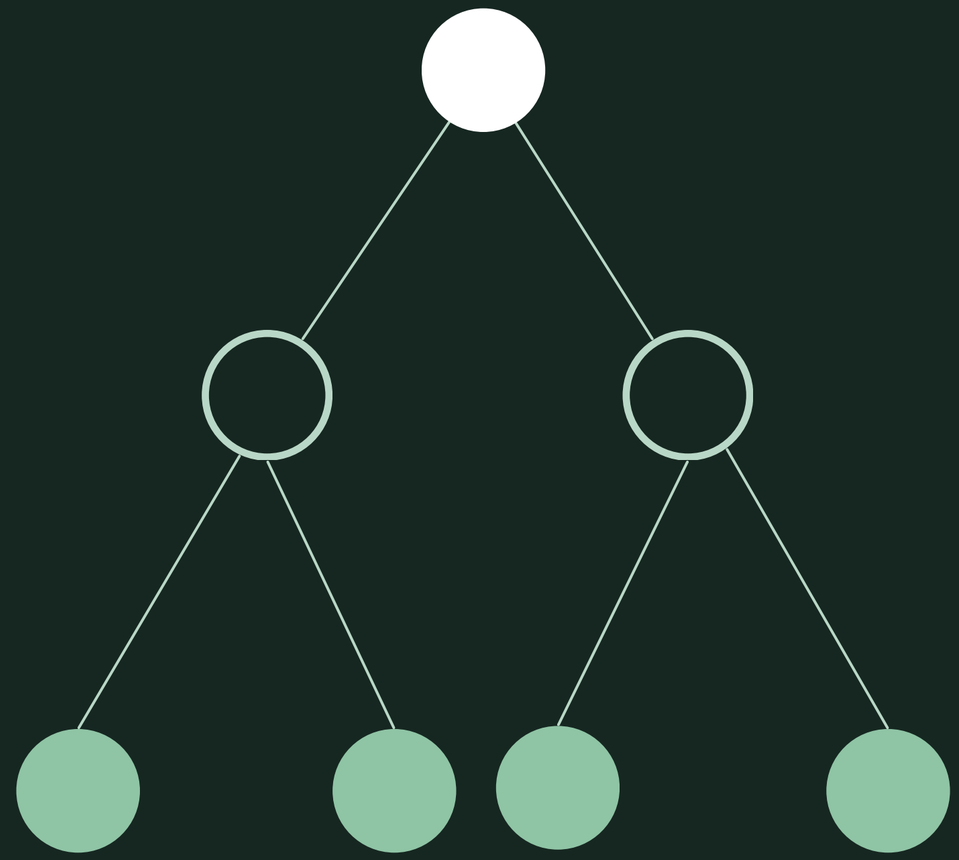
Decision trees

MLDS

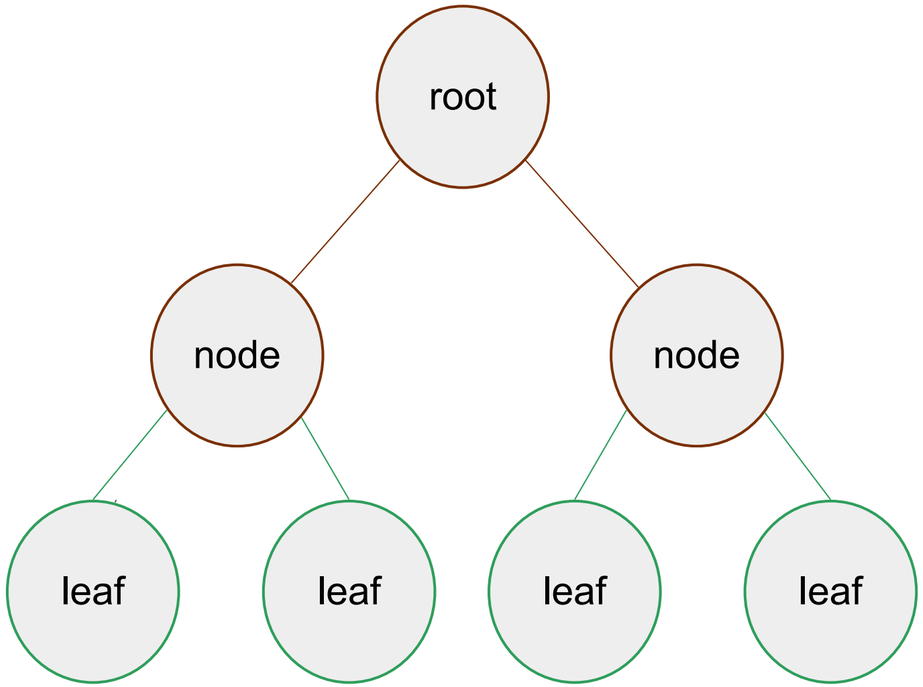


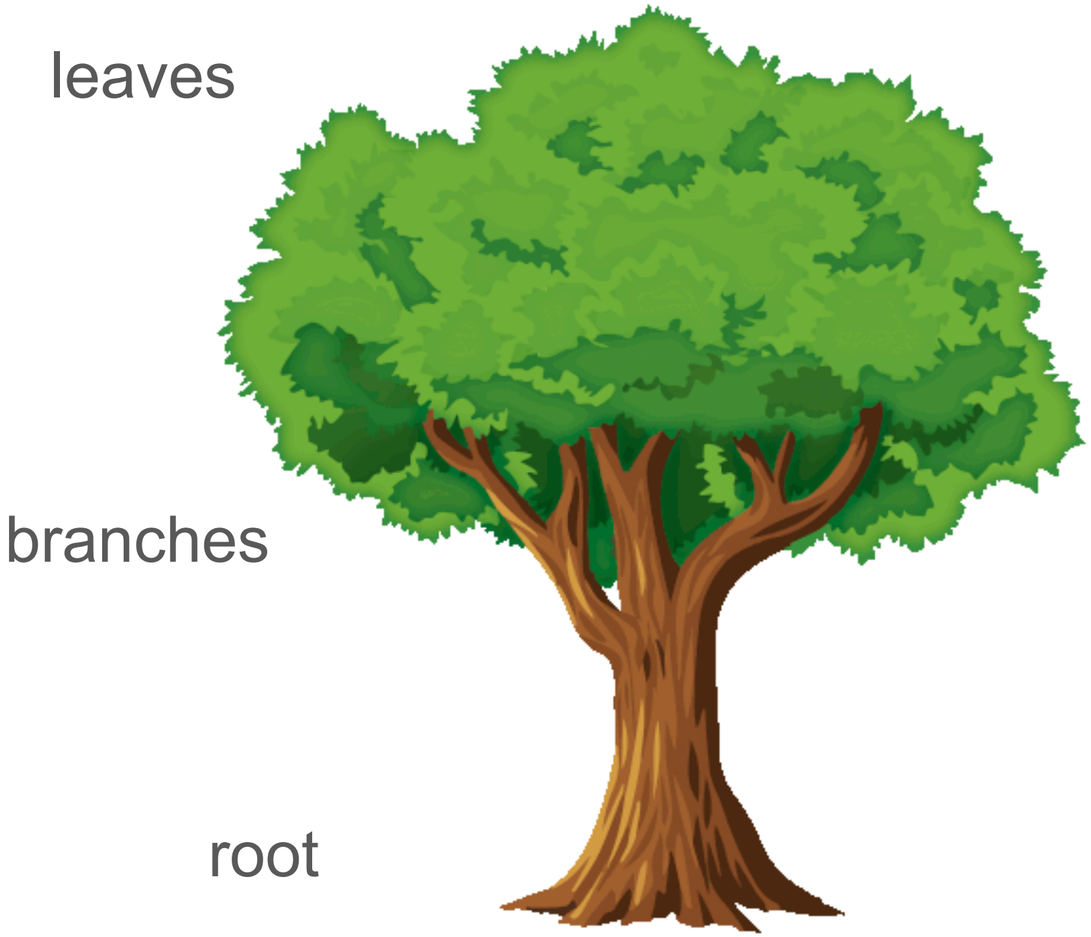
Tree-based methods

- Use a **tree-like structure** (hence the name tree-based) to learn from and make predictions on data.
- Can be used for **regression** (continuous target) and **classification** (binary target). We'll see why this works.
- **Common Methods:** Decision Trees, Random Forests, Gradient Boosting, XGBoost (Extreme Gradient Boosting)

What actually is a “tree”?

- A tree is a type of data structure with roots, nodes, and leaves.
- Why are trees upside-down? Computer scientists don’t see trees often because they don’t go outside.





A dummy example: club event attendance

Name	Free food?	Midterm week?	Days since RSVP	Showed up?
Anna	Yes	No	1	Yes
Bob	Yes	No	3	Yes
Chris	Yes	No	4	Yes
Dylan	Yes	No	5	Yes
Ashley	Yes	No	6	Yes
Fatima	Yes	Yes	2	No
Gabe	No	Yes	1	Yes
Hannah	No	No	3	No
Isaac	No	No	5	No
Jules	No	Yes	6	No
Kyrie	No	No	4	No
Louis	No	Yes	7	No

Features

Free food? Midterm week? → yes/no
Days since RSVP → number

Target

Showed up?
(yes/no → classification).

Goal: find a short list of yes/no questions that sorts the green rows from the red rows.

Step 1: pick the best first question

Everyone (12)



impurity 0.50
(as mixed as it gets)

Free food?

weighted impurity

0.28

Yes: 6 people



No: 6 people



← winner

Midterm week?

weighted impurity

0.44

Yes: 4 people



No: 8 people



Days since RSVP ≤ 1?

weighted impurity

0.40

Yes: 2 people



No: 10 people



How "best" is measured: Gini impurity

$$\text{Gini} = 1 - P(\text{show})^2 - P(\text{no-show})^2$$

0 = pure: everyone in the group has the same answer.

0.5 = a 50/50 coin flip (the worst case for yes/no).

A split's score = each group's Gini, weighted by group size. Lower wins.

Free food split: (6 × 0.28 + 6 × 0.28) / 12 = 0.28



5 show, 1 no-show

$1 - (5/6)^2 - (1/6)^2 = 0.28$



6 show, 6 no-show

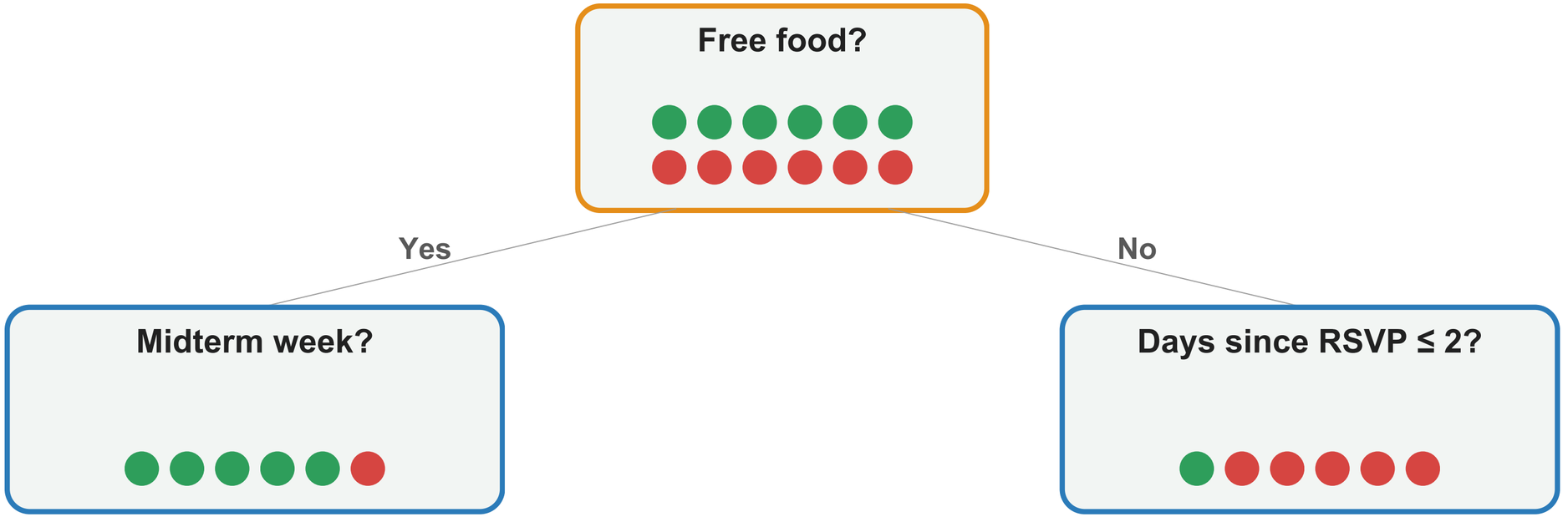
$1 - (½)^2 - (½)^2 = 0.50$



5 show, 0 no-show

$1 - 1^2 - 0^2 = 0$ (pure!)

Step 2: repeat on each branch

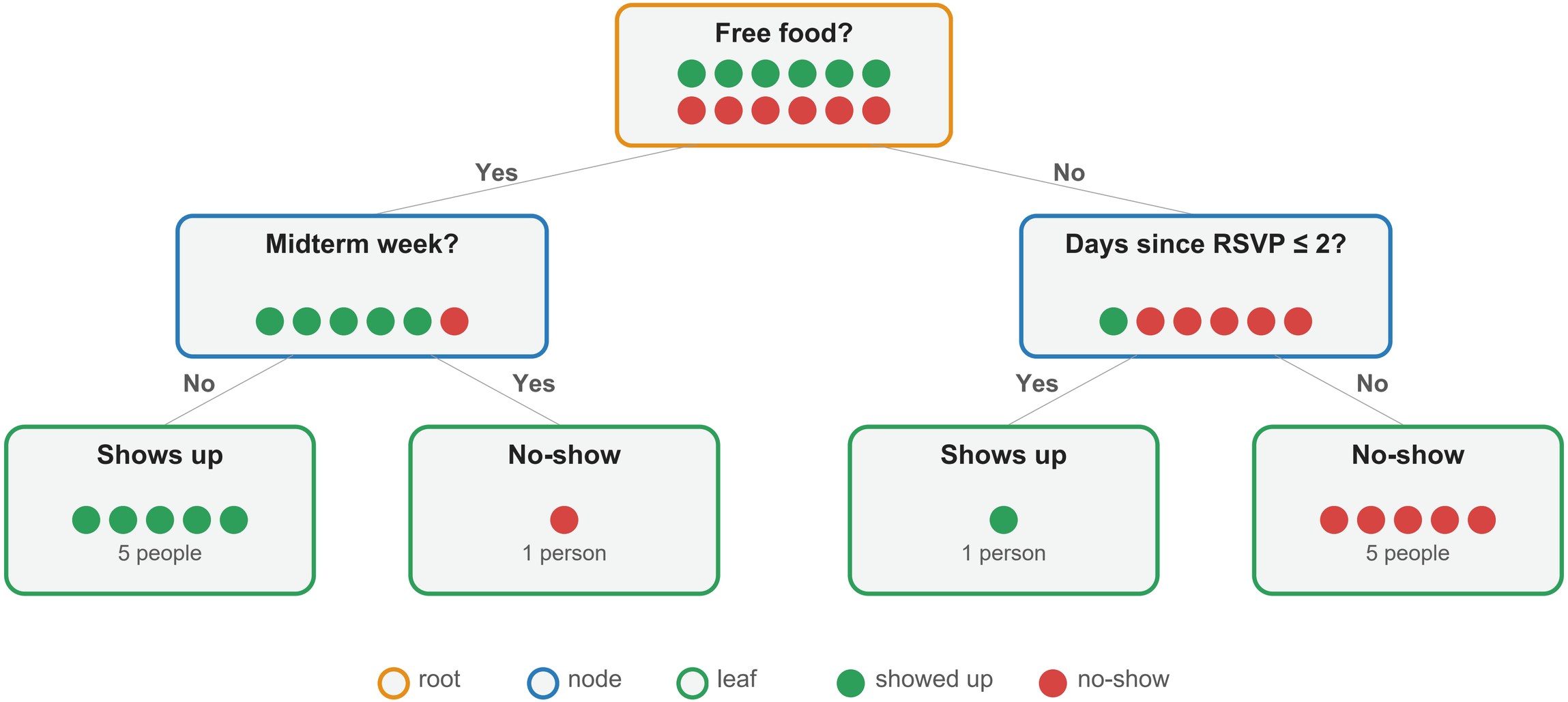


Of the 6 with food, only Fatima didn't come, and she had a midterm.

Of the 6 without food, only Gabe came, and he RSVP'd yesterday.

root node leaf showed up no-show

The finished tree

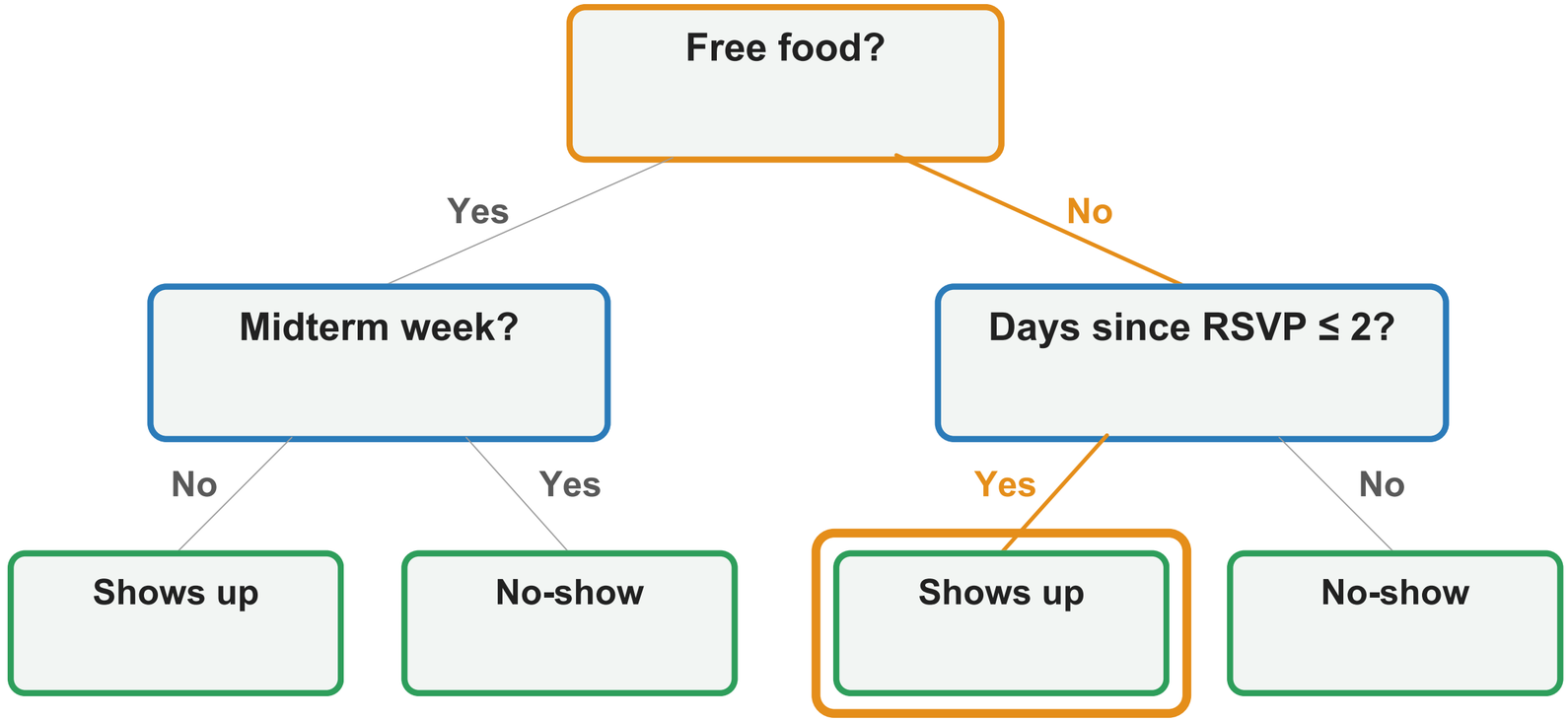


Predicting for someone new

New RSVP: Ines

Free food? **No**
Midterm week? **Yes**
Days since RSVP **1**

Prediction: shows up
The tree never asked about her midterm. It only checks the questions on *her* path.



How does a tree pick its own questions?

In our example, we picked the questions.
A real tree finds them by itself.

1. Try every possible question
2. Score each one: how well does it separate the people who came from the people who didn't?
3. Keep the best one, then do the same thing on each branch

That's the "learning" part: nobody writes the rules. The data picks them.

Every question it tries first

Our 12 RSVPs, lower score = cleaner split

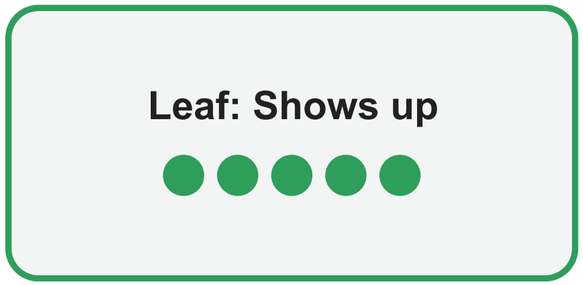
Free food?	← winner	**0.28**
Days since RSVP ≤ 1		**0.40**
Midterm week?		**0.44**
Days since RSVP ≤ 6		**0.45**
Days since RSVP ≤ 2		**0.48**
Days since RSVP ≤ 5		**0.48**
Days since RSVP ≤ 3		**0.49**
Days since RSVP ≤ 4		**0.49**

Nobody told it to ask about free food first. It figured that out from the data.

Same tree, two jobs: classification and regression

Classification

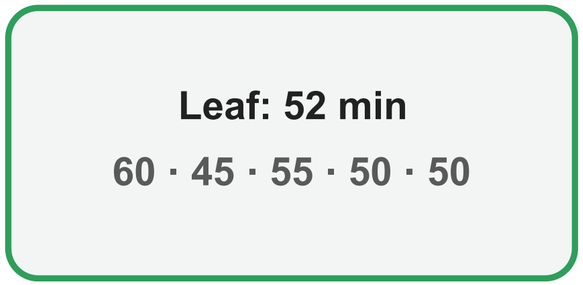
Target is a category (showed up: yes / no)



Leaf prediction = majority vote
5 green, 0 red → "shows up"
Share of the majority ≈ probability

Regression

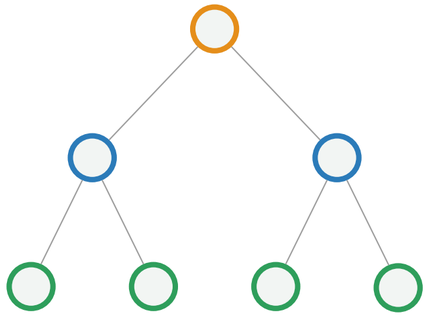
Target is a number (minutes stayed)



Leaf prediction = average of the leaf
60, 45, 55, 50, 50 → 52 minutes
Splits minimize **squared error**, not Gini

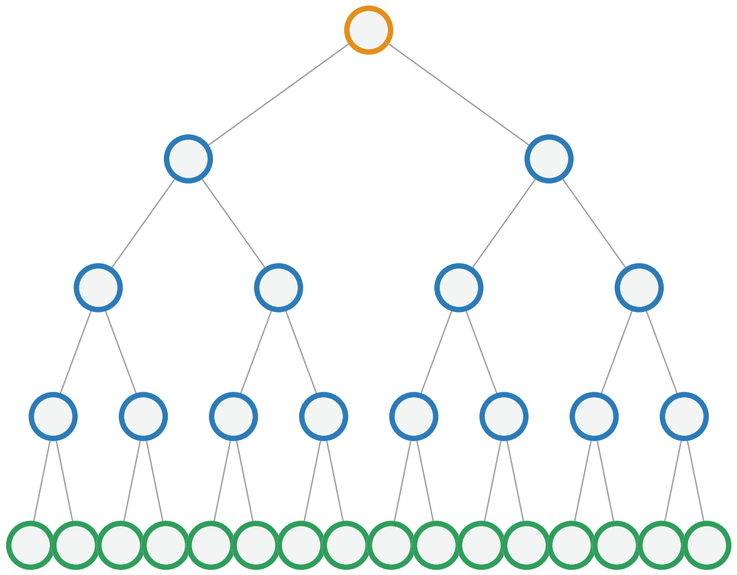
When do we stop splitting?

Shallow (depth 2)



simple rules, may miss patterns (underfit)

Deep (depth 4)



a leaf for every person, memorizes noise (overfit)

`max_depth`

How many questions in a row, at most

`min_samples_leaf`

Every leaf must hold at least this many rows.

`min_samples_split`

Don't split a group smaller than this.

Strengths and limitations

Strengths

- Readable (you can say the rules out loud)
- No scaling, no straight-line assumption
- Finds interactions on its own (food × midterm)
- Fast to train, fast to predict

Limitations

- Overfits: deep trees memorize noise
- Unstable: a few new rows can completely reshape the tree
- “Boxy”: smooth trends need lots of splits
- Greedy: one tree is rarely the most accurate

How it compares

Logistic Regression

Smoother and more stable, but you have to hand it interactions (feature selection). A tree finds them itself.

Random forest

Hundreds of trees on random samples, averaged. Overfits far less, but you lose the readable rules.

Gradient boosting (XGBoost)

Trees built one after another, each fixing the last one's mistakes. Usually most accurate on tables, hardest to explain.



Our data: hotel bookings

Hotel booking demand

119,390	32	2	37%
bookings (rows)	columns	hotels in Portugal	were canceled

Question we're answering

Will this booking be canceled?
Target: is_canceled (1 = canceled, 0 = not). Binary, so this is classification.

City Hotel (Lisbon): 79,330 bookings, 42% canceled. Resort Hotel (Algarve): 40,060, 28% canceled.
Arrivals from July 2015 to August 2017. Source: Antonio, Almeida & Nunes (2019), via TidyTuesday.

What's in each row?

Timing

```
lead_time
arrival_date_month
arrival_date_week_number
```

The stay

```
stays_in_weekend_nights
stays_in_week_nights
meal
reserved_room_type
```

Who's coming

```
adults
children
babies
customer_type
country
```

How it was booked

```
market_segment
distribution_channel
agent
company
```

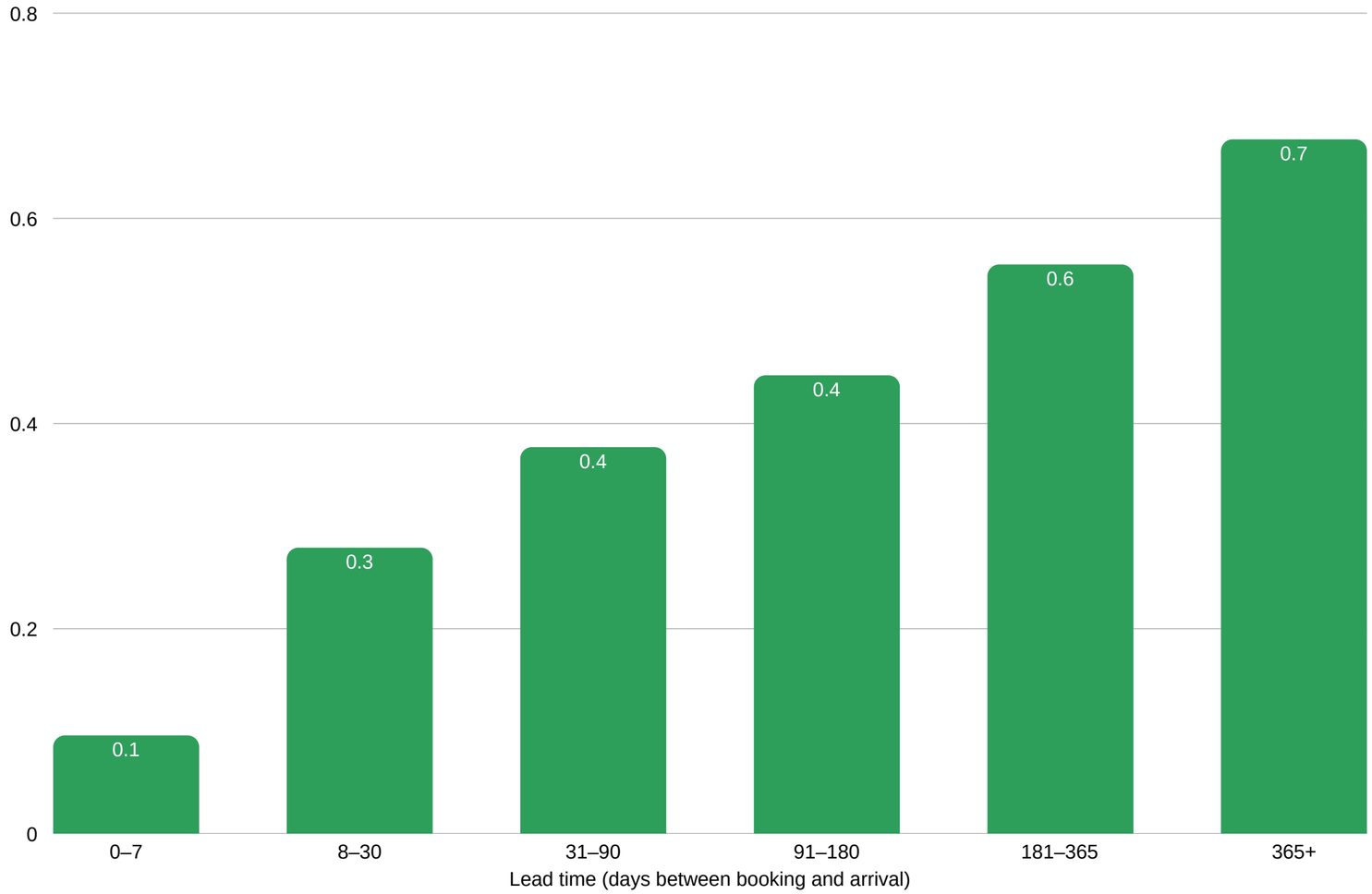
Guest history

```
is_repeated_guest
previous_cancellations
previous_bookings_not_canceled
```

Money and requests

```
adr
deposit_type
required_car_parking_spaces
total_of_special_requests
```

Bookings made far ahead cancel more



Other quick signals

0% of bookings that asked for a parking space canceled
48% with no special requests vs. 22% with one
42% vs 28%: City Hotel vs Resort Hotel

Watch out: columns that give away the answer

Drop before modeling

`reservation_status`

Canceled / No-Show = the target itself

`reservation_status_date`

set after the booking is canceled or completed

`assigned_room_type`

decided at check-in

`booking_changes`

counted until check-in or cancellation

Rule of thumb

Only use what you'd know at the moment the booking is made. If a column only exists because of the outcome, it's leakage.

Something to discuss

99.4% of non-refundable bookings were canceled. Real pattern or data quirk? The tree will lean on it either way.



The modeling process

From raw data to a model in 6 steps

1 Explore

shape, target balance, quick charts

→

2 Clean

drop leakage columns, fill missing values

→

3 Encode

one-hot text columns (no scaling needed!)

4 Split

80% train / 20% test, stratified

→

5 Fit

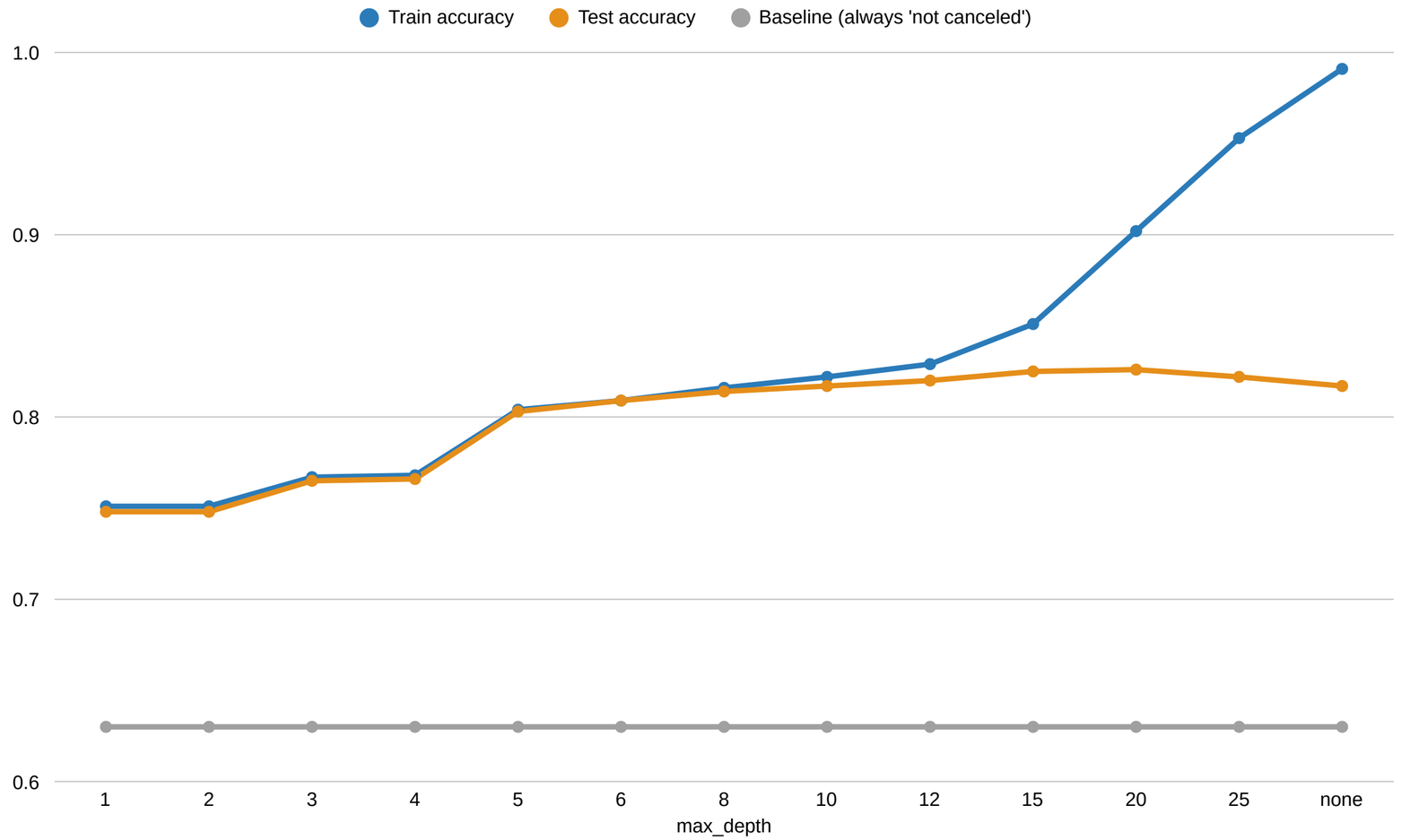
decision tree, try a few max_depth values

→

6 Evaluate

accuracy vs baseline, confusion matrix

Evaluate: deeper isn't always better



Unlimited depth

99% train

82% test

43 levels deep: it memorized the training set

Evaluate: what kinds of mistakes?

	Predicted: not canceled	Predicted: canceled
Actually not canceled	13,907	1,126
Actually canceled	3,312	5,533

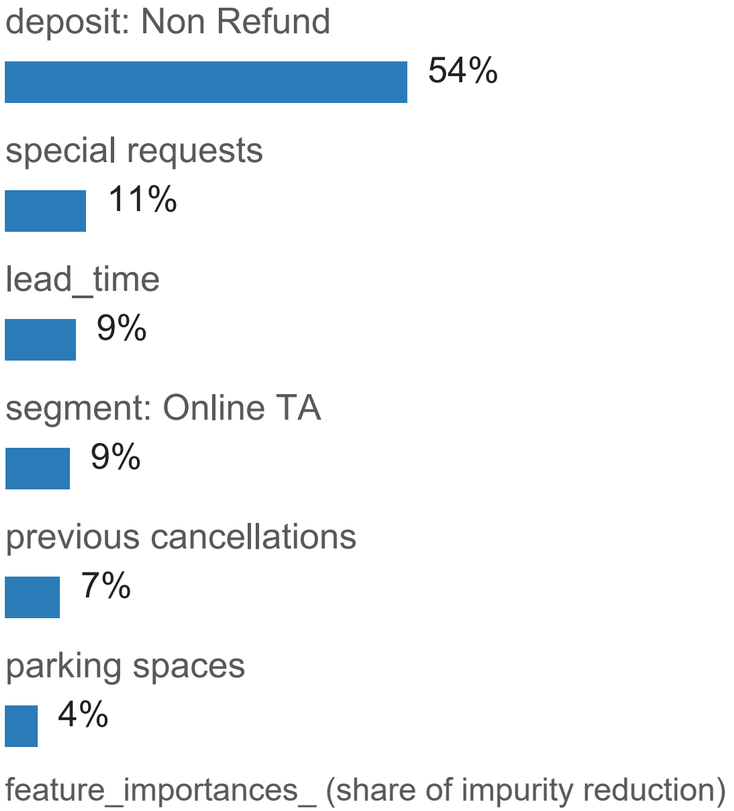
81%
accuracy

83%
precision

63%
recall

Depth-8 tree on 23,878 test bookings. It catches 63% of real cancellations, and when it flags one, it's right 83% of the time.

What the tree leans on



feature_importances_ (share of impurity reduction)